

Employee–Department–Project Schema

Aim

To create an Employee–Department–Project database schema, insert at least 30 employees across 5 departments and 8 projects, and execute SQL queries demonstrating selection, projection, aggregate functions, GROUP BY, HAVING, CASE expressions, and ORDER BY.

Software / Requirements

- MySQL / MySQL Workbench
- SQL database server
- Computer system with Windows/Linux/macOS

Database Design

The database contains four tables. Department stores department information, Employee stores employee information, Project stores project information, and Employee_Project represents the many-to-many relationship between employees and projects.

Table	Primary Key	Important Attributes
Department	Dept_ID	Dept_Name, Location
Employee	Emp_ID	Emp_Name, Job_Title, Salary, Hire_Date, Dept_ID
Project	Project_ID	Project_Name, Budget, Dept_ID
Employee_Project	Emp_ID + Project_ID	Hours_Worked

1. Create Database

```
CREATE DATABASE CompanyDB;  
  
USE CompanyDB;
```

2. Create Tables

Department Table

```
CREATE TABLE Department (  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(50) NOT NULL,  
    Location VARCHAR(50)  
);
```

Employee Table

```
CREATE TABLE Employee (  
    Emp_ID INT PRIMARY KEY,  
    Emp_Name VARCHAR(50) NOT NULL,  
    Job_Title VARCHAR(50),  
    Salary DECIMAL(10,2),  
    Hire_Date DATE,  
    Dept_ID INT,  
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
);
```

Project Table

```
CREATE TABLE Project (
    Project_ID INT PRIMARY KEY,
    Project_Name VARCHAR(100) NOT NULL,
    Budget DECIMAL(12,2),
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);
```

Employee_Project Table

```
CREATE TABLE Employee_Project (
    Emp_ID INT,
    Project_ID INT,
    Hours_Worked INT,
    PRIMARY KEY (Emp_ID, Project_ID),
    FOREIGN KEY (Emp_ID) REFERENCES Employee(Emp_ID),
    FOREIGN KEY (Project_ID) REFERENCES Project(Project_ID)
);
```

3. Insert Department Records

```
INSERT INTO Department VALUES
(1, 'Artificial Intelligence', 'Chandigarh'),
(2, 'Software Development', 'Bangalore'),
(3, 'Cyber Security', 'Delhi'),
(4, 'Data Science', 'Pune'),
(5, 'Human Resources', 'Mumbai');
```

4. Insert 30 Employee Records

```
INSERT INTO Employee VALUES
(101, 'Aarav Sharma', 'AI Engineer', 85000, '2022-01-15', 1),
(102, 'Priya Verma', 'ML Engineer', 92000, '2021-06-20', 1),
(103, 'Rohan Gupta', 'AI Developer', 78000, '2023-03-10', 1),
(104, 'Ananya Singh', 'Research Engineer', 105000, '2020-08-12', 1),
(105, 'Karan Mehta', 'ML Engineer', 88000, '2022-11-05', 1),
(106, 'Neha Kapoor', 'AI Analyst', 72000, '2024-01-18', 1),

(107, 'Rahul Kumar', 'Software Engineer', 75000, '2022-02-14', 2),
(108, 'Sneha Patel', 'Backend Developer', 82000, '2021-09-21', 2),
(109, 'Aditya Jain', 'Frontend Developer', 70000, '2023-05-17', 2),
(110, 'Simran Kaur', 'Full Stack Developer', 95000, '2020-11-10', 2),
(111, 'Vivek Mishra', 'Software Engineer', 78000, '2022-07-25', 2),
(112, 'Ishita Rao', 'QA Engineer', 65000, '2024-02-11', 2),

(113, 'Arjun Malhotra', 'Security Engineer', 90000, '2021-04-16', 3),
(114, 'Megha Joshi', 'Cyber Security Analyst', 82000, '2022-10-09', 3),
(115, 'Yash Thakur', 'Security Consultant', 98000, '2020-06-13', 3),
(116, 'Pooja Nair', 'SOC Analyst', 68000, '2023-08-19', 3),
(117, 'Dev Patel', 'Security Engineer', 87000, '2022-12-01', 3),
(118, 'Tanvi Shah', 'Ethical Hacker', 91000, '2021-03-22', 3),

(119, 'Akash Singh', 'Data Scientist', 100000, '2020-09-15', 4),
(120, 'Kriti Sharma', 'Data Analyst', 76000, '2022-05-12', 4),
(121, 'Mohit Agarwal', 'Data Scientist', 95000, '2021-11-18', 4),
(122, 'Riya Das', 'Data Analyst', 72000, '2023-04-07', 4),
(123, 'Nikhil Bansal', 'ML Data Scientist', 89000, '2022-08-30', 4),
(124, 'Sakshi Roy', 'BI Analyst', 74000, '2024-03-14', 4),

(125, 'Manish Verma', 'HR Manager', 85000, '2019-07-10', 5),
```

```
(126, 'Divya Mehta', 'HR Executive', 60000, '2022-01-25', 5),
(127, 'Suresh Kumar', 'Recruiter', 58000, '2023-06-18', 5),
(128, 'Ayesha Khan', 'HR Executive', 62000, '2021-10-11', 5),
(129, 'Varun Gupta', 'Training Manager', 73000, '2020-12-05', 5),
(130, 'Nandini Sharma', 'HR Analyst', 65000, '2024-04-20', 5);
```

5. Insert 8 Project Records

```
INSERT INTO Project VALUES
(201, 'AI Recommendation System', 1500000, 1),
(202, 'Computer Vision Platform', 2000000, 1),
(203, 'E-Commerce Platform', 2500000, 2),
(204, 'Mobile Banking Application', 1800000, 2),
(205, 'Cyber Threat Detection', 2200000, 3),
(206, 'Security Monitoring System', 1700000, 3),
(207, 'Customer Analytics Platform', 1900000, 4),
(208, 'Employee Management System', 1200000, 5);
```

6. Assign Employees to Projects

```
INSERT INTO Employee_Project VALUES
(101, 201, 120),
(102, 201, 150),
(103, 202, 100),
(104, 201, 180),
(105, 202, 140),
(106, 202, 90),

(107, 203, 160),
(108, 203, 180),
(109, 204, 130),
(110, 203, 200),
(111, 204, 150),
(112, 204, 100),

(113, 205, 170),
(114, 205, 140),
(115, 206, 190),
(116, 206, 120),
(117, 205, 160),
(118, 206, 150),

(119, 207, 200),
(120, 207, 140),
(121, 207, 180),
(122, 207, 120),
(123, 207, 160),
(124, 207, 100),

(125, 208, 180),
(126, 208, 140),
(127, 208, 120),
(128, 208, 150),
(129, 208, 160),
(130, 208, 110);
```

7. SQL Queries

A. Selection

Selection retrieves rows that satisfy a specified condition.

Query 1: Employees earning more than ₹90,000

```
SELECT *  
FROM Employee  
WHERE Salary > 90000;
```

Query 2: Employees belonging to Department 1

```
SELECT *  
FROM Employee  
WHERE Dept_ID = 1;
```

B. Projection

Projection retrieves selected columns from a table.

Query 3: Display employee names and salaries

```
SELECT Emp_Name, Salary  
FROM Employee;
```

Query 4: Display employee name and job title

```
SELECT Emp_Name, Job_Title  
FROM Employee;
```

C. Aggregate Functions

Aggregate functions perform calculations over multiple rows.

Query 5: Average salary

```
SELECT AVG(Salary) AS Average_Salary  
FROM Employee;
```

Query 6: Highest salary

```
SELECT MAX(Salary) AS Highest_Salary  
FROM Employee;
```

Query 7: Lowest salary

```
SELECT MIN(Salary) AS Lowest_Salary  
FROM Employee;
```

Query 8: Total salary expenditure

```
SELECT SUM(Salary) AS Total_Salary  
FROM Employee;
```

Query 9: Total number of employees

```
SELECT COUNT(*) AS Total_Employees
FROM Employee;
```

D. GROUP BY

GROUP BY groups rows having the same value in specified columns.

Query 10: Count employees in each department

```
SELECT d.Dept_Name, COUNT(e.Emp_ID) AS Employee_Count
FROM Department d
JOIN Employee e ON d.Dept_ID = e.Dept_ID
GROUP BY d.Dept_ID, d.Dept_Name;
```

Query 11: Average salary of each department

```
SELECT d.Dept_Name,
       AVG(e.Salary) AS Average_Salary
FROM Department d
JOIN Employee e ON d.Dept_ID = e.Dept_ID
GROUP BY d.Dept_ID, d.Dept_Name;
```

E. HAVING

HAVING filters groups after GROUP BY has been applied.

Query 12: Departments having more than 5 employees

```
SELECT Dept_ID, COUNT(*) AS Employee_Count
FROM Employee
GROUP BY Dept_ID
HAVING COUNT(*) > 5;
```

Query 13: Departments with average salary above ₹80,000

```
SELECT d.Dept_Name,
       AVG(e.Salary) AS Average_Salary
FROM Department d
JOIN Employee e ON d.Dept_ID = e.Dept_ID
GROUP BY d.Dept_ID, d.Dept_Name
HAVING AVG(e.Salary) > 80000;
```

F. CASE Expression

CASE provides conditional logic within a SQL expression.

Query 14: Categorize employees by salary

```
SELECT Emp_Name,
       Salary,
       CASE
           WHEN Salary >= 90000 THEN 'High Salary'
           WHEN Salary >= 70000 THEN 'Medium Salary'
           ELSE 'Low Salary'
       END AS Salary_Category
FROM Employee;
```

Query 15: Categorize employees by department

```
SELECT Emp_Name,  
       Dept_ID,  
       CASE  
         WHEN Dept_ID = 1 THEN 'AI'  
         WHEN Dept_ID = 2 THEN 'Software'  
         WHEN Dept_ID = 3 THEN 'Cyber Security'  
         WHEN Dept_ID = 4 THEN 'Data Science'  
         WHEN Dept_ID = 5 THEN 'HR'  
         ELSE 'Other'  
       END AS Department_Name  
FROM Employee;
```

G. ORDER BY

ORDER BY sorts the result set in ascending or descending order.

Query 16: Employees ordered by salary ascending

```
SELECT Emp_Name, Salary  
FROM Employee  
ORDER BY Salary ASC;
```

Query 17: Employees ordered by salary descending

```
SELECT Emp_Name, Salary  
FROM Employee  
ORDER BY Salary DESC;
```

Query 18: Employees ordered by department and salary

```
SELECT Emp_Name, Dept_ID, Salary  
FROM Employee  
ORDER BY Dept_ID ASC, Salary DESC;
```

8. Output

Execute the above queries in MySQL Workbench or the assigned SQL environment. Attach/paste screenshots of the query execution and result tables in this section according to the lab-file requirements.

Output:

Emp_ID	Emp_Name	Job_Title	Salary	Hire_Date	Dept_ID
102	Priya Verma	ML Engineer	92000.00	2021-06-20	1
104	Ananya Singh	Research Engineer	105000.00	2020-08-12	1
110	Simran Kaur	Full Stack Developer	95000.00	2020-11-10	2
115	Yash Thakur	Security Consultant	98000.00	2020-06-13	3
118	Tanvi Shah	Ethical Hacker	91000.00	2021-03-22	3
119	Akash Singh	Data Scientist	100000.00	2020-09-15	4
121	Mohit Agarwal	Data Scientist	95000.00	2021-11-18	4
Emp_ID	Emp_Name	Job_Title	Salary	Hire_Date	Dept_ID
101	Aarav Sharma	AI Engineer	85000.00	2022-01-15	1
102	Priya Verma	ML Engineer	92000.00	2021-06-20	1
103	Rohan Gupta	AI Developer	78000.00	2023-03-10	1
104	Ananya Singh	Research Engineer	105000.00	2020-08-12	1
105	Karan Mehta	ML Engineer	88000.00	2022-11-05	1
106	Neha Kapoor	AI Analyst	72000.00	2024-01-18	1

Output:

Emp_Name	Salary
Aarav Sharma	85000.00
Priya Verma	92000.00
Rohan Gupta	78000.00
Ananya Singh	105000.00
Karan Mehta	88000.00
Neha Kapoor	72000.00
Rahul Kumar	75000.00
Sneha Patel	82000.00
Aditya Jain	70000.00
Simran Kaur	95000.00
Vivek Mishra	78000.00
Ishita Rao	65000.00
Arjun Malhotra	90000.00
Megha Joshi	82000.00
Yash Thakur	98000.00
Pooja Nair	68000.00
Dev Patel	87000.00
Tanvi Shah	91000.00
Akash Singh	100000.00
Kriti Sharma	76000.00

Output:

Emp_Name	Job_Title
Aarav Sharma	AI Engineer
Priya Verma	ML Engineer
Rohan Gupta	AI Developer
Ananya Singh	Research Engineer
Karan Mehta	ML Engineer
Neha Kapoor	AI Analyst
Rahul Kumar	Software Engineer
Sneha Patel	Backend Developer
Aditya Jain	Frontend Developer
Simran Kaur	Full Stack Developer
Vivek Mishra	Software Engineer
Ishita Rao	QA Engineer
Arjun Malhotra	Security Engineer
Megha Joshi	Cyber Security Analyst
Yash Thakur	Security Consultant
Pooja Nair	SOC Analyst
Dev Patel	Security Engineer
Tanvi Shah	Ethical Hacker
Akash Singh	Data Scientist

Average_Salary
80333.333333
Highest_Salary
105000.00
Lowest_Salary
58000.00
Total_Salary
2410000.00

Dept_Name	Employee_Count
Artificial Intelligence	6
Software Development	6
Cyber Security	6
Data Science	6
Human Resources	6

Dept_Name	Average_Salary
Artificial Intelligence	86666.666667
Software Development	77500.000000
Cyber Security	86000.000000
Data Science	84333.333333
Human Resources	67166.666667

Dept_Name	Average_Salary
Artificial Intelligence	86666.666667
Cyber Security	86000.000000
Data Science	84333.333333

Emp_Name	Salary	Salary_Category
Aarav Sharma	85000.00	Medium Salary
Priya Verma	92000.00	High Salary
Rohan Gupta	78000.00	Medium Salary
Ananya Singh	105000.00	High Salary
Karan Mehta	88000.00	Medium Salary
Neha Kapoor	72000.00	Medium Salary
Rahul Kumar	75000.00	Medium Salary
Sneha Patel	82000.00	Medium Salary
Aditya Jain	70000.00	Medium Salary
Simran Kaur	95000.00	High Salary
Vivek Mishra	78000.00	Medium Salary
Ishita Rao	65000.00	Low Salary

Emp_Name	Salary
Suresh Kumar	58000.00
Divya Mehta	60000.00
Ayesha Khan	62000.00
Nandini Sharma	65000.00
Ishita Rao	65000.00
Pooja Nair	68000.00
Aditya Jain	70000.00
Neha Kapoor	72000.00
Riya Das	72000.00
Varun Gupta	73000.00
Sakshi Roy	74000.00
Rahul Kumar	75000.00
Kriti Sharma	76000.00
Vivek Mishra	78000.00
Rohan Gupta	78000.00
Sneha Patel	82000.00
Megha Joshi	82000.00
Manish Verma	85000.00
Aarav Sharma	85000.00

9. Result

The Employee–Department–Project database schema was successfully created. Records for 30 employees, 5 departments, and 8 projects were inserted. SQL queries demonstrating selection, projection, aggregate functions, GROUP BY, HAVING, CASE expressions, ORDER BY, and employee–project relationships were successfully executed.